Installing AskAI

Installing AskAI

This guide takes you from the downloaded zip to a working setup on Windows, macOS, Linux or WSL: what to prepare, how to preview and install, how to check it worked, and every file AskAI creates. It takes about ten minutes. What each setting does and why: `docs/ARCHITECTURE.md`.

The short version

AskAI installs straight from GitHub: nothing to download first. Run **one command per project**. It fetches AskAI into `~/.claude/askai/src` (so `askai upgrade` can refresh it later), installs or refreshes it for Claude Code, Codex, Antigravity and Cursor with every optimiser (the `max` profile), writes the project's knowledge pack, and ends with the list of things only you can do. Running it again for another project is safe: whatever is already set up is skipped.

Windows (PowerShell):

```
& ([scriptblock]::Create((irm
https://raw.githubusercontent.com/aalokshah-business/AskAi/main/get.ps
1))) -Target all -Workflow max -Project C:\code\myapp -Yes
```

macOS, Linux or WSL:

```
curl -fsSL
https://raw.githubusercontent.com/aalokshah-business/AskAi/main/get.sh
| bash -s -- --target all --profile max --project ~/code/myapp --yes
```

Then work through `askai todo <repo>` (sign-ins, restarts, app settings, project details) and check everything with `askai infra <repo>`. Add `-DryRun` / `--dry-run` for a preview, or leave out the `max` profile for the evidence-based default.

Or ask your agent to do it. Paste this into Claude Code, Codex, Antigravity or Cursor, opened in the project:

```
Set up AskAI for this repository. From the repository root, run the
one command for this OS and wait for it to finish (it asks no
questions):
- macOS, Linux or WSL: curl -fsSL
https://raw.githubusercontent.com/aalokshah-business/AskAi/main/get.sh
| bash -s -- --target all --profile max --project . --yes
- Windows: powershell -NoProfile -ExecutionPolicy Bypass -Command "&
([scriptblock]::Create((irm
https://raw.githubusercontent.com/aalokshah-business/AskAi/main/get.ps
1))) -Target all -Workflow max -Project . -Yes"
```

```
Then run: askai todo .   (if askai is not found yet, use its full
path: ~/.claude/askai/bin/askai, or askai.cmd on Windows)
Reply with what was installed, anything that failed, and my to-do
list. Change no other files. Remind me to restart this session
afterwards.
```

Other ways to get it. Clone it (`git clone https://github.com/aalokshah-business/AskAi ~/tools/AskAI`) or download the ZIP from GitHub (Code, Download ZIP), then run the installer in that folder with the same options: `bash ~/tools/AskAI/install.sh --target all --profile max --project ~/code/myapp --yes`, or on Windows `powershell -ExecutionPolicy Bypass -File C:\tools\AskAI\install.ps1 -Target all -Workflow max -Project C:\code\myapp -Yes`. With Node.js and git installed, `npx github:aalokshah-business/AskAi install --target all --profile max --project . --yes` works too.

Keeping it current. `askai upgrade` fetches the latest AskAI from GitHub and re-applies your profile; `askai update` upgrades the tools it installed. Before running a script from the internet you can read it first: get.sh, get.ps1, and SECURITY.md for what the installer changes.

The rest of this guide explains each step, every option and every file.

1. What you get

- The `askai` command in every new terminal (`askai check`, `askai ask`, `askai break`, `askai resume`, `askai usage` and the rest; `askai help <command>`).
- Token-saving settings and short rules in each tool you use: Claude Code, Codex, Antigravity and Cursor.
- A handoff command inside each tool: `/askai-handoff` (Codex: `$askai-handoff`).
- For each repository you choose: shared instruction files, an ignore list for Cursor, and a local `.ai-work/` folder for handoffs and measurements.

Nothing runs in the background: AskAI adds no hooks. Settings you already chose are never changed.

2. Before you start

Need	Windows 10/11	macOS	Linux and WSL
Node.js 18 or newer	`winget install OpenJS.NodeJS.LTS` (install.ps1 offers it)	`brew install node` (install.sh offers it)	nvm (`https://github.com/nvm-sh/nvm`) or your distribution's package
Package manager for tools	winget (built into Windows 11; "App Installer" from the Microsoft Store on 10)	Homebrew recommended	Homebrew optional; otherwise the vendors' installer scripts and cargo
Shell	PowerShell 5.1 or	zsh or bash	bash

Need	Windows 10/11	macOS	Linux and WSL
	later		
git	Recommended (the ledger records branch and commit)	Recommended	Recommended
Network	claude.ai, astral.sh (uv, for Serena and markitdown), github.com, registry.npmjs.org, pypi.org	same	same

Install the AI tools you use first, or let AskAI install Claude Code. Codex: `npm i -g @openai/codex`. Antigravity: its installer from antigravity.google. Cursor: cursor.com/download. AskAI sets up every one it finds. Context Mode (opt-in only) needs Node 22.5 or newer.

AskAI never uses sudo or administrator rights; everything goes into your user folders.

3. Environment setup

Sign in with your plan, not an API key.

Tool	Sign in	Confirm
Claude Code	`claude`, then `/login`	`/status` shows your Pro, Max or Team plan
Codex	`codex`, then sign in with ChatGPT	`codex login status`
Antigravity	`agy`, then sign in with Google	`/usage` shows per-model quota
Cursor	Sign in in the editor	Dashboard, Usage

Remove billing variables unless you mean to pay per token: an API key overrides your plan, and a base URL sends Claude Code through a proxy.

```
# Windows (PowerShell): remove for your user, then for this window
foreach ($v in
'ANTHROPIC_API_KEY','ANTHROPIC_BASE_URL','OPENAI_API_KEY') {
  [Environment]::SetEnvironmentVariable($v, $null, 'User'); Remove-
Item "Env:$v" -ErrorAction SilentlyContinue }

# macOS, Linux, WSL: unset now, then delete any export lines for them
from ~/.zshrc, ~/.bashrc or ~/.profile
unset ANTHROPIC_API_KEY ANTHROPIC_BASE_URL OPENAI_API_KEY
env | grep -E 'ANTHROPIC|OPENAI'      # should print nothing
```

`askai check` and `askai doctor` flag these variables, including an `ANTHROPIC_API_KEY` set inside Claude Code's `settings.json`.

Proxies. `HTTPS_PROXY` and `HTTP_PROXY` are fine for downloads. Do not set `ANTHROPIC_BASE_URL` for a proxy unless you mean to leave your plan's billing.

Custom config folders. If you keep a tool's settings outside the default folder, pass it to AskAI: `--claude-home`, `--codex-home`, `--agy-home` or `--cursor-home` (PowerShell: `-ClaudeHome`, `-CodexHome`, `-AgyHome`, `-CursorHome`).

Windows: unblock the download. Files from a downloaded zip can be blocked from running. In the unzipped folder:

```
Get-ChildItem -Recurse | Unblock-File
```

4. Get and verify the package

Unzip the AskAI ZIP (from GitHub: Code, Download ZIP) and open a terminal in its folder. Check that no file was changed:

```
sha256sum -c SHA256SUMS.txt --quiet          # Linux, WSL
shasum -a 256 -c SHA256SUMS.txt --quiet      # macOS

# Windows: prints nothing when every file matches
Get-Content SHA256SUMS.txt | ForEach-Object { $h, $f = $_ -split '\
s+', 2
  if ((Get-FileHash $f -Algorithm SHA256).Hash -ne $h.ToUpper()) {
"MISMATCH $f" } }
```

Verify before opening files in an editor: re-saving can change line endings.

5. Preview

A preview shows every change and writes nothing.

```
powershell -ExecutionPolicy Bypass -File .\install.ps1 -DryRun     #
Windows

bash install.sh --dry-run                                         #
macOS, Linux, WSL
```

Each line names a component, its state and what will happen. `ok` means already right and skipped. `partial` means some settings are missing and will be added. `missing` means it will be installed. `other` means it does not apply here, for example a tool that is not installed. Add `-Target all` / `--target all` to see all four tools even before they are installed.

6. Install

```
powershell -ExecutionPolicy Bypass -File .\install.ps1             #
Windows

bash install.sh                                                   #
macOS, Linux, WSL
```

AskAI asks once, then:

1. Copies itself to `~/.claude/askai/app` and writes the `askai` command to `~/.claude/askai/bin`. On Windows it adds that folder to your user PATH. On macOS and Linux, `install.sh` offers to add one line to `~/.zshrc` or `~/.bashrc`.
2. Installs each component in order: `token-kit` first, then the Claude Code and Cursor components, then Codex's and Antigravity's, then the handoff commands and the remaining Claude Code settings.
3. Backs up every file before its first change (`~/.claude/askai/backups/`) and records what it added (`~/.claude/askai/manifest.json`).
4. Prints a summary with next steps.

Choose tools and work type.

Option (PowerShell / bash)	Effect
`-Target all` / `--target all`	Claude Code, Codex, Antigravity and Cursor. The default `auto` sets up Claude Code plus every other tool found
`-Target claude,codex` / `--target claude,codex`	Only those tools
`-Workflow minimal` / `--profile minimal`	Settings, rules, status line and handoff only
`-Workflow coding` / `--profile coding`	The default: adds Serena for code navigation
`-Workflow docs` / `--profile docs`	Adds markitdown for PDF and Office files
`-Workflow full` / `--profile full`	Both, plus ccusage
`-Workflow max` / `--profile max`	Everything: full plus RTK, Context Mode, Headroom (MCP mode), Graphify, Context7, the Caveman plugin, LSP plugins (ts, py) and the base tools, in all four tools
`-Yes` / `--yes`	Do not ask

Re-running is safe: components already right are skipped.

7. Check it worked

Open a new terminal, so the PATH change applies, then:

```
askai version        # 4.2.1
askai status         # each component: ok, partial, missing or other
askai check          # every guideline: ok, fix or FAIL, with the
```

```
command that fixes it
askai doctor          # broken config and billing traps
```

Then start a new session in each tool:

Tool	Look for
Claude Code	`/status` shows your plan; `/model` shows Haiku if you had no default; `/context` shows CLAUDE.md with the askai block loaded
Codex	`/status`; `~/.codex/config.toml` has effort `medium` and verbosity `low`
Antigravity	`/usage`; `~/.gemini/GEMINI.md` has the askai block
Cursor	`~/.cursor/rules/` holds the `askai-*.mdc` files

8. Set up each repository

```
askai init ~/code/myapp            # asks what the project is about,
then writes the knowledge pack; never overwrites
askai check ~/code/myapp --fix     # check it, and fix what can be
fixed
askai infra ~/code/myapp           # everything set up for the four
tools and this repository
```

Or during install: `bash install.sh --project ~/code/myapp` (PowerShell: `-Project C:\code\myapp`).

`askai init` writes `AGENTS.md` (the router every agent reads, under 333 lines), `CLAUDE.md`, `GEMINI.md`, `docs/ai/PROJECT_MAP.md`, `ARCHITECTURE.md`, `FEATURES.md`, `WORK_ORDER.md`, `MODELS.md`, the `architect`, `builder` and `runner` subagents for Claude Code, Codex, Cursor and Antigravity, four skills, `.cursorignore` and `.ai-work/`. Afterwards fill in the Purpose column of `PROJECT_MAP.md` (or ask the agent to use the architecture-map skill). The table below lists the files that need your input.

File	What to do
`AGENTS.md`	Fill in the stack, the test, build and lint commands, and where things live. Keep it under 60 lines: every agent reads it
`CLAUDE.md`	Imports `@AGENTS.md`; add only Claude-specific notes
`docs/ai/PROJECT_PROFILE.md`	Longer project notes, read only when needed
`.cursorignore`	Keeps build output, lockfiles and `.env` out of Cursor

File	What to do
`.ai-work/`	Handoff, ledger and measurements; gitignored automatically

8a. Daily commands

Moment	Command
Start a task	`askai go claude --tier build "fix the cart rounding in src/cart.ts"` (also codex, cursor, agy)
Shape a dictated ask	`askai prompt "..." --copy`
Things only you can do	`askai todo` (per tool and repository; `askai todo done <id>` marks one)
Every command in full	`askai help all`; one command: `askai help <command>`
How to use a plugin	`askai plugins info <id>`: commands in a terminal and inside the agent, how to check it, its savings, how to switch it off
What each plugin saved	`askai savings` (runs rtk gain, ccusage, headroom stats and the rest for you)
Check etiquette	`askai work`, `askai etiquette`, `askai guide <moment>`, `askai ask <words>`
Coming back	`askai continue` (same as `askai resume`)
Review before handing off	`askai diff`; architecture at a glance: `askai graph`
Before /clear or a break	`/askai-handoff`, then `askai clear` or `askai break`
Coming back	`askai resume --copy`
See waste	`askai audit`; usage: `askai usage 7`
Switch MCP servers	`askai mcp off <name>` before the next session

9. Optional extras

Option	Effect
`--lsp ts,py` (also kotlin, ruby, swift)	Claude Code's official code-intelligence plugins, plus the npm

Option	Effect
	language servers for ts and py
`--main-model sonnet`	Sonnet instead of Haiku as Claude Code's default, when you have none set
`--codex-model NAME`	A cheaper default model for Codex
`--ccusage`, `--markitdown`	The usage reporter; the document converter
`--lean-git`	Also remove Claude Code's git instructions from every turn, if you never use it for commits
`--caveman lite|full|ultra|off`	How terse the replies are (default full)
`--headroom`, `--graphify`, `--context7` (PowerShell `-Headroom`, `-Graphify`, `-Context7`)	One optimiser at a time, wired into every tool in play. Headroom is set up in MCP mode only; its proxy mode is never set up automatically because it can move Claude Code to API billing
`--base-tools` (PowerShell `-BaseTools`)	ripgrep, jq and gh where missing: agents search and inspect with them far more cheaply
`--rtk`, `--context-mode`	Opt-in only: independent benchmarks found they cost more

10. Files AskAI creates

Paths are under your home folder (`%USERPROFILE%` on Windows).

Tool	Files
AskAI	`.claude/askai/app/` (the program), `.claude/askai/bin/askai` (the command), `.claude/askai/manifest.json`, `.claude/askai/backups/`, `.claude/askai/sessions/` (status-line snapshots), `.claude/askai/cache/`
Claude Code	`.claude/settings.json` (added keys only), `.claude/CLAUDE.md` (a marked block), `.claude/rules/askai-*.md`, `.claude/commands/askai-*.md`
Codex	`.codex/config.toml` (added keys

Tool	Files
	only), `.codex/AGENTS.md` (a marked block), `.agents/skills/askai-handoff/`
Antigravity	`.gemini/antigravity-cli/settings.json`, `.gemini/GEMINI.md` (a marked block), `.gemini/config/rules/askai-*.md`, the `askai-handoff` skill in `.gemini/antigravity-cli/skills/` and `.gemini/config/skills/`
Cursor	`.cursor/rules/askai-*.mdc`, `.cursor/commands/askai-handoff.md`
Coding profile	Serena as an MCP server in each tool (`.claude.json`, `.cursor/mcp.json`, `.codex/config.toml`, `.gemini/config/mcp_config.json`)

The full list, with every key and value: `docs/ARCHITECTURE.md`, sections 8 and 9.

11. AskAI and the standalone script

`standalone/token-setup.mjs` applies the same settings without AskAI, for a machine where you only want the configuration. Use one or the other, never both: running both would load the token rules twice.

- **token-setup ran first:** `askai install` takes over. Its `token-kit` step runs `token-setup --uninstall` (with backups), then AskAI applies the same settings under its own manifest. Your own text is kept.
- **AskAI is installed:** token-setup stops, points to `askai check`, and changes nothing (`--force` overrides).
- A test applies both in separate folders and confirms AskAI sets every setting token-setup sets, with the same values.

12. Update, uninstall, restore

```
askai update                  # upgrade the tools AskAI installed and refresh its rules
askai uninstall --dry-run     # see what would be removed
askai uninstall               # remove only what AskAI added (add --tools to remove installed tools too)
```

```
askai restore                  # list backups; askai restore <file>
puts one back
```

13. Troubleshooting the install

Symptom	Fix
`node` not found	Install Node.js 18+ (section 2), open a new terminal, re-run
`winget` not found (Windows 10)	Install "App Installer" from the Microsoft Store
No Homebrew on macOS	AskAI uses the vendors' installer scripts instead; ripgrep is then a manual step it names
"running scripts is disabled" or "not digitally signed"	Use `-ExecutionPolicy Bypass` as shown, and run `Get-ChildItem -Recurse | Unblock-File`
`askai` not recognised	Open a new terminal; on macOS and Linux accept the PATH line, or add `~/.claude/askai/bin` to PATH
A component failed	Read its line in the summary, fix the cause, re-run; components already done are skipped
Permission denied	AskAI writes only in your home folder: check who owns it; never run it with sudo
`/status` in Claude Code shows API billing	Remove the billing variables (section 3), restart the terminal
Antigravity spent paid credits	`askai check`; in the app set AI Credit Overages to Never

14. What was tested

- 204 automated tests in throwaway folders. Every component installs, detects and uninstalls; settings are never overridden; `install.ps1`'s options and actions match askai.js; AskAI applies every token-setup setting; a token-setup run is taken over cleanly; a full install followed by uninstall leaves nothing of AskAI in any of the four tools.
- A real run of `install.sh` on Linux into a throwaway home folder: install, the installed `askai` command, and uninstall.
- Previews of every OS and tool combination (`docs/PREVIEW_MATRIX.md`): Windows and macOS were simulated on Linux by making Node report those systems.
- Not yet run on real machines: `install.ps1` itself (PowerShell was not available where AskAI was built and tested) and live installs of each tool. On each new machine, run the preview first, then `askai check` after installing.

Windows self-test. One command runs the real `install.ps1` in a temporary folder (your own settings are never touched) and prints a summary to paste back:

```
powershell -ExecutionPolicy Bypass -File .\scripts\windows-selftest.ps1
```

It checks the checksums, runs `install.ps1 -DryRun` (must write nothing), installs, runs `askai check`, uninstalls (must leave nothing), runs the automated tests, and deletes the temporary folder.